

高速 SPI · IP 清单（首目录）

可复用 RTL 放在仓库根路径；具体上板/仿真工程见 Projects/。

1. 异步 FIFO — `async_fifo.v`
2. SPI 从机 — `spi_slave_controller.v` + `spi_slave_shift.v`
3. SPI 主机 — `spi_master.v` (PLL 预留)
4. 字节 FIFO 桥 — `spi_fifo_interface.v`
5. 验证核心 — `spi_verify_top.v`
6. UART TX / RX — `uart_tx.v`、`uart_rx.v`
7. 伪 ADC — `triangle_gen.v` (三角)、`sine_gen.v` (正弦 LUT)
- 7b. 同步 FIFO — `sync_fifo.v` (同时钟 UART 推流用)
8. UART 波形核心 — `uart_wave_core.v` (自发推流)
9. UART 回波核心 — `uart_echo_core.v` (上位机下发再回传)
10. 实验系统核心 — `lab_sys_core.v` (`adc_clk` 伪 ADC → `async_fifo` → SPI → UART)
11. 上板工程
 - Projects/`spi_verify/` — SPI 主从回环自检 (看 LED, 无上位机)
 - Projects/`uart_wave/` — UART 回波显象
 - Projects/`lab_sys/` — 片内理想回环 (金参考)
 - Projects/`lab_sys_ext/` — SCLK/MOSI 飞线非理想 (复现 SPI 错)
12. SDRAM 控制器
13. 仲裁器及控制电路

板卡原厂资料: Gowin/ (引脚表、手册; 非工程顶层)。

Projects ↔ 上位机 Python

依赖 (一次即可) :

```
cd host
pip3 install -r requirements.txt
```

串口: 板载 WCH, 常用 `/dev/cu.usbserial-20230814131` (先 `--list` 确认)。
不要同时开 WCHSerial / 其它串口助手, 否则会抢口。

工程	已下载的顶层	调用	作用
Projects/lab_sys/	lab_sys	python3 lab_sys_plot.py -p <□> -b 115200	收伪 ADC→SPI→UART 帧并画图
Projects/uart_wave/	uart_wave (echo)	python3 uart_wave_echo.py -p <□> -b 115200	PC 下发复杂波，FPGA 回传对比
Projects/uart_wave/	若改回自发推流	python3 uart_wave_plot.py -p <□> -b 115200	只收帧画图
(调试)	任意推流工程	python3 uart_wave_listen.py -p <□> -b 115200	只打印字节/帧
Projects/spi_verify/	spi_verify	—	无 py；LED / 板级自检

示例：

```
cd host
python3 lab_sys_plot.py --list
python3 lab_sys_plot.py --port /dev/cu.usbserial-20230814131 --baud 115200
```

帧格式：

- uart_wave: AA 55 | SEQ | N | DATA[N] | XOR
- lab_sys: AA 55 | SEQ | N | ERR | DATA[N] | XOR (多 ERR；XOR 含 ERR)

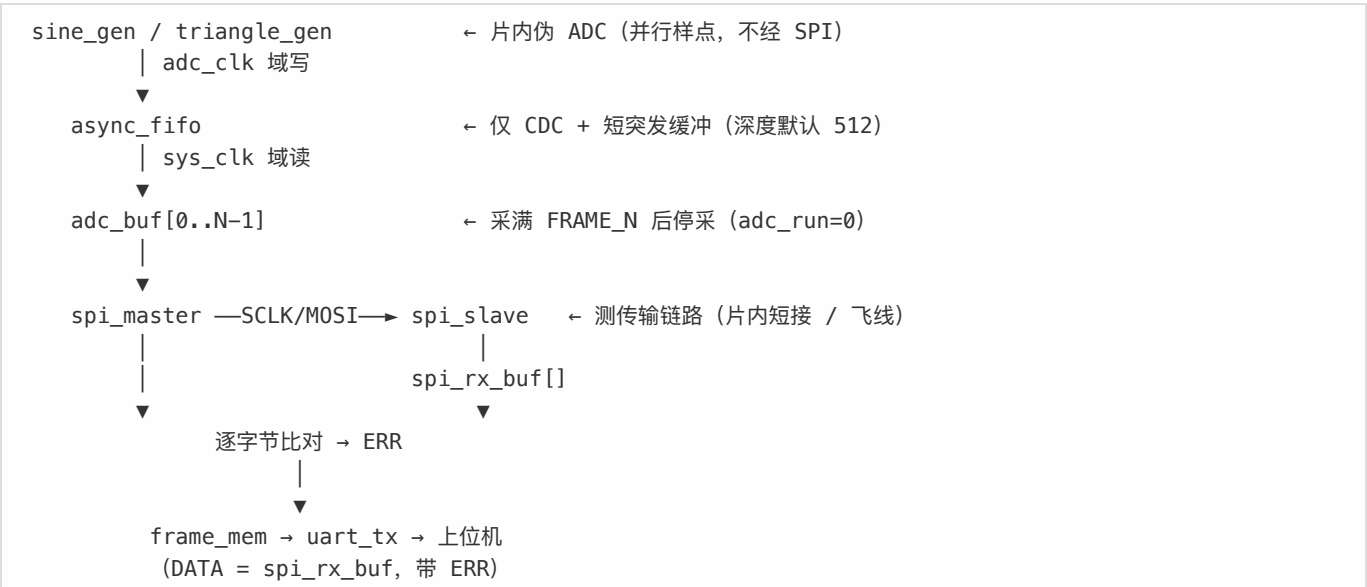
脚本与工程必须匹配：下了 lab_sys 不要跑 uart_wave_echo.py。

lab_sys 架构（当前体系）

数据路径（纠正常见误解）

不是「伪 ADC → SPI 进 FPGA → FIFO → UART 直读 FIFO」。

真实路径：



块	作用
伪 ADC	已知波形样点，代替真 ADC 并行输出
async_fifo	adc_clk→sys_clk 跨时钟；不是长录存储
adc_buf	本拍发送前金参考
SPI 主从	把金参考整段搬一遍，看链路是否错比特
spi_rx_buf	从机收到的样点（可能已损伤）
UART	慢速倒出一帧快照；非连续实时流

默认：adc_clk≈1 MHz，FRAME_N=128，SPI_CLK_DIV=4→sclk≈6.25 MHz，BAUD=115200。

工程分层

文件	角色
lab_sys_core.v	整条流水线 + FSM
Projects/lab_sys/lab_sys.v	板级顶层；spi*_o 片内接 spi*_i
Projects/lab_sys_ext/lab_sys_ext.v	同核；飞线 G15→G14(SCLK)、G16→H15(MOSI)；CS 片内

相关 RTL（按链路顺序）

sine_gen.v / triangle_gen.v

adc_clk 上按 SAMPLE_HZ 出 sample_stb+8bit sample。正弦=相位+64 点 LUT；三角=折返计数。WAVE_SEL 在 core 里 generate 二选一；enable=0 停采。

async_fifo.v

双时钟格雷码指针。lab_sys: wclk=adc_clk, rclk=sys_clk, 深度 AFIFO_DEPTH。满则 core 不写；读用 rd_en, 下一拍取 rdata。

spi_master.v

自产 sclk/cs_n/mosi, Mode0。start+xfer_bytes=N 发整帧；tx_data←adc_buf[spi_tx_idx], byte_done 推进索引。CLK_DIV: sclk 周期=2*CLK_DIV 个 eng_clk。

spi_slave_controller.v

sclk/mosi/cs_n 双触发同步到 clk, 译 sample_stb/shift_stb/cs_*/mosi_s。只做边沿门控，不移位。

spi_slave_shift.v

按脉冲累加 RX / 移 TX；满字节→byte_done+rx_data。lab_sys 中 TX 固定 0, miso 回 master。

uart_tx.v

tx_start 发 1 字节；tx_busy 握手。core 用 ISSUE→WBUSY→WIDLE 防连打丢字节。

lab_sys_core.v

IDLE→FLUSH→SAMP→SPI→CMP→BUILD→ISSUE/WBUSY/WIDLE（详见 A1-notes.md）。

旁路模块 (*lab_sys* 未用)

文件	用途
<code>spi_fifo_interface.v</code>	SPI 字节口↔双 <code>async_fifo</code> ；通用桥， <code>lab_sys</code> 未挂
<code>spi_verify_top.v</code>	LED 自检； <code>Projects/spi_verify/</code>
<code>sync_fifo.v</code> / <code>uart_wave_core.v</code> / <code>uart_echo_core.v</code> / <code>uart_rx.v</code>	UART 推流/回波； <code>Projects/uart_wave/</code>

缓冲与后续 *DDR*

浅 FIFO 只扛 CDC + 采 N 突发；采满停写。加长捕捉：

ADC → 浅 `async_fifo(CDC)` → *DDR3*(深存) → 再读出做 SPI/UART

DDR 扩的是 **adc_buf** 容量，不是「塞在 UART 与浅 FIFO 之间代替 CDC」。